ECE 220 Computer Systems & Programming

Lecture 1 — Memory-Mapped I/O

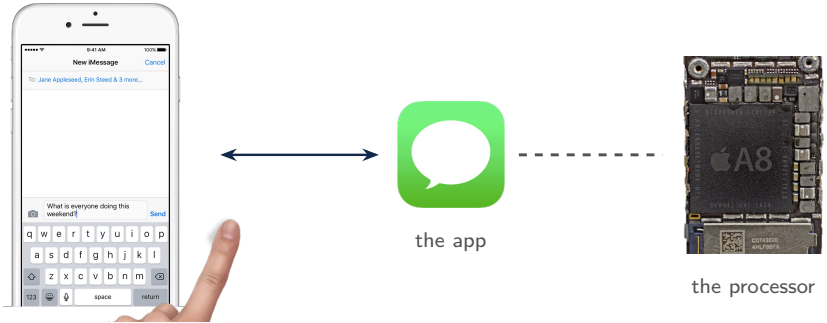
August 25, 2026

Prof. Sayan Mitra mitras@illinois.edu
Course website: courses.grainger.illinois.edu/ece220/fa2026

ILLINOIS
Electrical & Computer Engineering
GRAINGER COLLEGE OF ENGINEERING

The I/O Problem

How do we get data from the outside world into the computer — and back out?



the outside world

the app

the processor

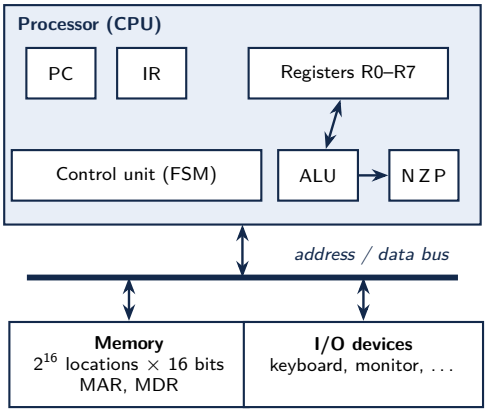
Today's Two Questions

Your finger is a billion times slower than the processor — and neither knows when the other is ready.

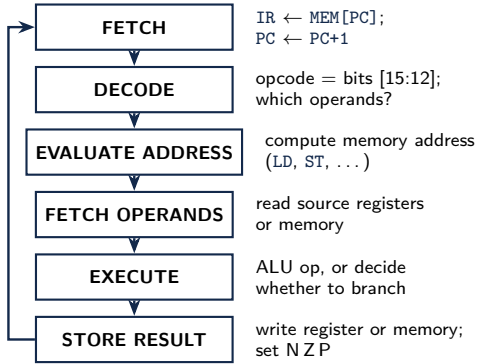
1. How does a keystroke actually *reach* the processor?
2. How does the processor *know* a new keystroke has arrived?

We will arrive at the answer to both by the end of today.

LC-3 Review — Architecture and the Instruction Cycle



Instruction cycle (repeat forever):



LC-3 Review — Instruction Set Architecture (ISA)

Data type: 16-bit 2's-complement integers

Addressing modes (how the location of an operand is specified):

- Non-memory addresses: immediate (part of the instruction), register
- Memory addresses: PC-relative, base+offset, indirect

Opcodes (16-bit instructions; bits [15:12] specify the opcode):

- *Operate* instructions: `ADD`, `AND`, `NOT`
- *Data movement* instructions: `LD`, `LDI`, `LDR`, `LEA`, `ST`, `STR`, `STI`
- *Control* instructions: `BR`, `JSR`/`JSRR`, `JMP`, `RET`, `TRAP`, `RTI`
- Condition codes: N (negative), Z (zero), P (positive)

LC-3 Review — Using LD, LDI, LDR, LEA

```
; LC-3 review: LD, LDI, LDR, LEA
.ORIG x3000
    LD   R6, LABEL
    LDI  R6, LABEL
    LDR  R2, R6, #1
    LEA  R2, LABEL
LABEL   .FILL x5001
.END
```

Assume the following memory:

Address	Data
x5001	x6001
...	...
x6001	x7001
x6002	x7002

What ends up in the registers?

LD: R6 ← M[_____] = _____

LDI: R6 ← M[_____] = _____

LDR: R2 ← M[_____] = _____

LEA: R2 ← _____

LC-3 Warm-up

1. Initialize a register to zero
2. Copy a value from R0 to R1
3. Compute $8 - 2$

In-Class Problem: Multiplication

Write a program to perform the multiplication 5×4.

- Need a way to store 5 and 4 as arguments
- There is **no multiply instruction** in LC-3 — so we repeat addition
- Register plan:

  `R0 - result, init to 0`
  `R1 - multiplicand, init to 5`
  `R2 - loop counter, init to 4`

```
.ORIG x3000
; R0 result, R1 multiplicand, R2 counter

; initialize: R0 = 0, R1 = 5, R2 = 4

; loop: exit when the counter is zero

;   add the multiplicand to the result

;   decrement the counter and repeat

DONE HALT
.END
```

Course Logistics

- Lecture (section BL1): T/R, 12:30–1:50 pm CT, ECEB 1002 (except exam days)
- Discussion sections (labs) on Fridays
  10 makeup pts/lab worksheet toward MPs (except MP12)
- **MPs**: due Thursdays at 10 pm CT; 100 pts each; late penalty 2 pts/hour; 12 MPs, lowest score (except MP12) dropped
- **Quizzes**: 6 programming quizzes, in person at CBTF; lowest dropped
- **Exams**: 2 midterms + final exam (paper format, in person)
- DRES accommodations and religious observances: follow the instructions on the course website
- Academic integrity: violations lead to FAIR cases

Grading	
MPs	15%
CBTF quizzes	20%
Midterm exams	$2 \times 20\%$
Final exam	25%

Textbook: Patt & Patel, *Introduction to Computing Systems: From Bits & Gates to C/C++ & Beyond*, 2nd or 3rd edition

Tools & Resources

- **Course website**: all relevant course info, MP write-ups, etc.
- **Canvas**: gradebook
- **GitHub**: MP/lab code release and submission (**individual**)
- **Gradescope**: lab worksheets (extra credit) and MP regrades
- **Ed**: discussion board monitored by TAs
- **CBTF** (PrairieTest): quiz reservation tool
- **PrairieLearn**: practice quizzes and CBTF testing platform
- **Resources**: CARE, Counseling Center, DRES

Tips for Success in ECE 220

Keep up with the pace

Every lecture builds on the previous one.

Do the MPs yourself

The quizzes and exams check that you can.

Manage your time well

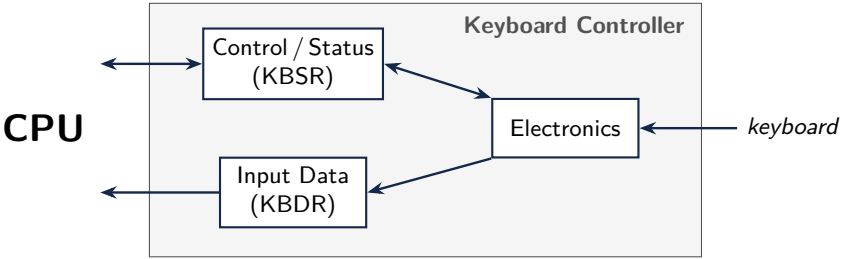
Start MPs early; 10 pm Thursday comes fast.

The I/O Problem, Revisited

Back to I/O: the keyboard and the processor live in different worlds.

- How to get data from a device (keyboard) to the processor?
- The producer of data (finger on a keyboard) works much, *much* more slowly than the consumer of that data (processor)
- We need to account for ***asynchrony*** operation: I/O device and processor run at their own speeds
- We will use a simple **producer/consumer handshake**: the device *signals* when it has (or is ready for) data

I/O Controller



Control/Status registers

- CPU **tells** the device what to do: **write** to the control register
- CPU **checks** whether a new keystroke has arrived: **read** the status register

Data registers

- CPU transfers data to/from the device — here, **reads** the character

Device electronics

- Performs the actual operation (detects the keystroke, latches the character)

Memory-Mapped I/O

- Assign a **memory address** to each device register
- Use data movement instructions (load/store) for control and data transfer

LC-3 input and output device registers

_____ stores the **ASCII value** of the character entered from the **keyboard**

_____ lets the processor know a new value has been entered

_____ stores the **ASCII value** of the character to be displayed on the **monitor**

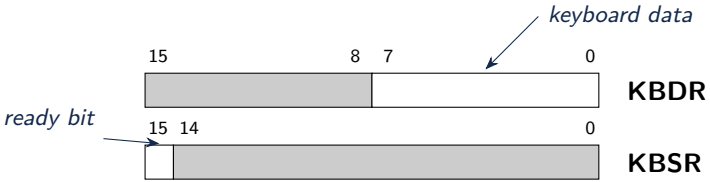
_____ lets the processor know the display is ready for a new value

LC-3 Memory-Mapped Device Registers

Address	Contents	Comments
x0000		; system space
...		
x3000		; user space:
		; programs and data
...		
xFE00	KBSR	; keyboard status
xFE02	KBDR	; keyboard data
xFE04	DSR	; display status
xFE06	DDR	; display data
...		
xFFFF		

- These are the memory **addresses** to which the device registers (KBDR, etc.) are **mapped**
- But the device registers are physically **separate** from the memory
- Memory-mapping device registers is a very common way to design interfaces for computing systems

Read from the Keyboard — Handshaking Using KBDR/KBSR



KBSR[15] controls the synchronization of the slow keyboard and the fast processor.

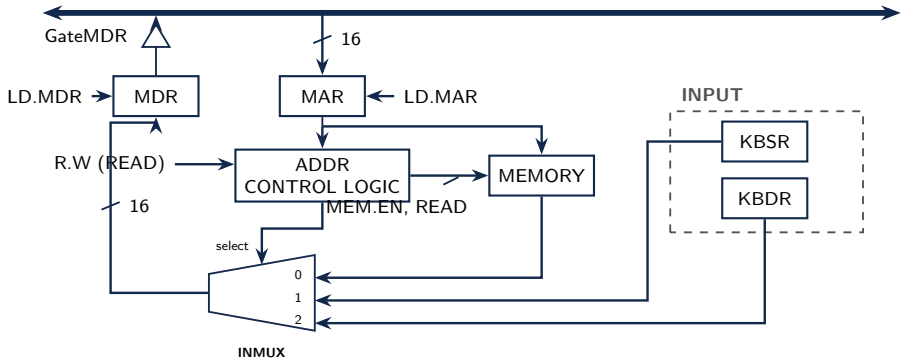
When a key on the keyboard is struck

- _____ is placed in KBDR[7:0] (upper bits are zero)
- _____ (KBSR[15]) is automatically set to 1 (by the keyboard circuit)
- Keyboard is _____ (new character entries will be ignored)

When KBDR is read

- _____ is automatically set to 0 (by the keyboard circuit)
- Keyboard is _____

Circuit for Memory-Mapped Input

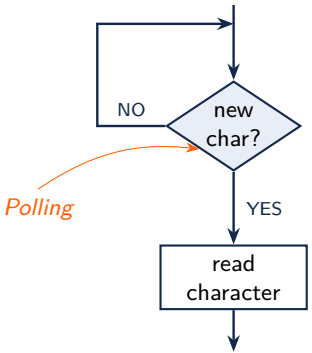


Conventional read `LD R0, addr`: MAR ← addr; MDR ← Mem[MAR]; R0 ← MDR

Memory-mapped input `LD R0, xFE02`: MAR ← xFE02; MDR ← _____; R0 ← MDR

The address decides where the data comes from — xFE02 selects the keyboard, not memory.

Read from the Keyboard — Basic LC-3 Routine



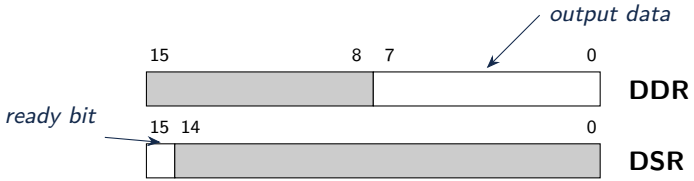

```
.ORIG x3000
; set up a loop to check the ready bit in KBSR

; branch to the beginning if no keyboard input

; otherwise, load the data from KBDR into R0

        HALT
KBSR_ADDR   .FILL   xFE00
KBDR_ADDR   .FILL   xFE02
.END
```

Output to the Monitor — Handshaking Using DDR/DSR



DSR[15] controls the synchronization of the fast processor and the slow monitor display.

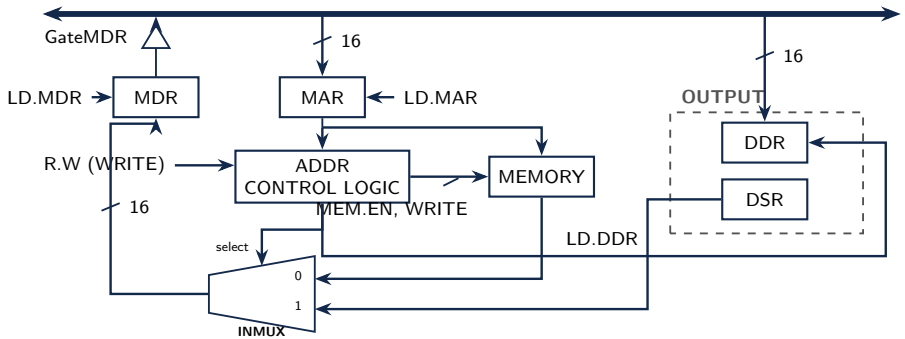
When the monitor is ready to display a new character

- _____ (DSR[15]) is automatically set to 1 (by the monitor circuits)

When data is written to DDR

- _____ is automatically set to 0 (by the monitor circuits), and the character in _____ is displayed
- Any other character data written to DDR is _____ while DSR[_____] is zero

Circuit for Memory-Mapped Output

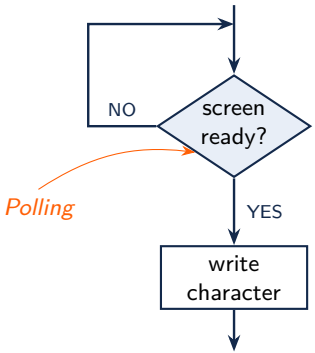


Conventional write `ST R0, addr`: MAR ← addr; MDR ← R0; Mem[MAR] ← MDR

Memory-mapped output `ST R0, xFE06`: MAR ← xFE06; MDR ← R0; _____ ← MDR

Asynchrony again: when is it *safe* to store? Poll the ready bit — the code, next.

Output to the Monitor — Basic LC-3 Routine




```
.ORIG x3000
        LD      R0, CHAR        ; character to display

; set up a loop to check the ready bit in DSR

; branch back to the beginning if the display is not ready

; otherwise, store the data from R0 into DDR

        HALT
CHAR        .FILL   x0041       ; 'A'
DSR_ADDR    .FILL   xFE04
DDR_ADDR    .FILL   xFE06
.END
```

Echo Routine Implementation

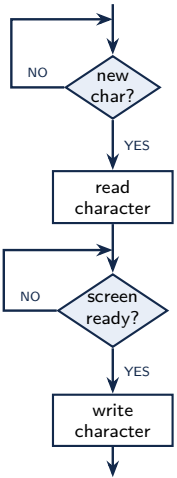
```
.ORIG x3000
; poll the keyboard: wait for a new character

; read the character from KBDR into R0

; poll the display: wait until it is ready

; write the character from R0 to DDR

        HALT
KBSR_ADDR    .FILL    xFE00
KBDR_ADDR    .FILL    xFE02
DSR_ADDR     .FILL    xFE04
DDR_ADDR     .FILL    xFE06
.END
```



Summary & What's Next

Today

- **The I/O problem**: devices and the CPU run at different speeds, asynchronously — and neither knows when the other is ready
- **The idea**: memory-mapped device registers (KBSR xFE00, KBDR xFE02, DSR xFE04, DDR xFE06) plus a ready-bit **handshake**
- **The mechanics**: the address control logic steers loads/stores to the device registers; **polling** loops busy-wait on the ready bit — the same pattern for **keyboard**, then **display**, then both (echo)

Next lecture

- TRAPs: asking the operating system to do I/O for us

Code from today: `mult.asm`, `kb_poll.asm`, `out_poll.asm`, `echo.asm` in the course code repository.